

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : To study and Implement divide and conquer technique			
EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/03		ISSUE NO. : 00	ISSUE DATE : 01.07.2023	
REV. DATE :		REV. NO. :	DEPTT. : INFORMATION TECHNOLOGY	
LABORATORY : Design and Analysis of Algorithm			SEMESTER : VI	PAGE: 1 OF 6

1.0) AIM:

To study and Implement divide and conquer technique.

2.0) SCOPE:

- To Implement Divide and Conquer Algorithmic Strategy.
- To implement Quick Sort Algorithm.

3.0) FACILITIES/ APPARATUS:

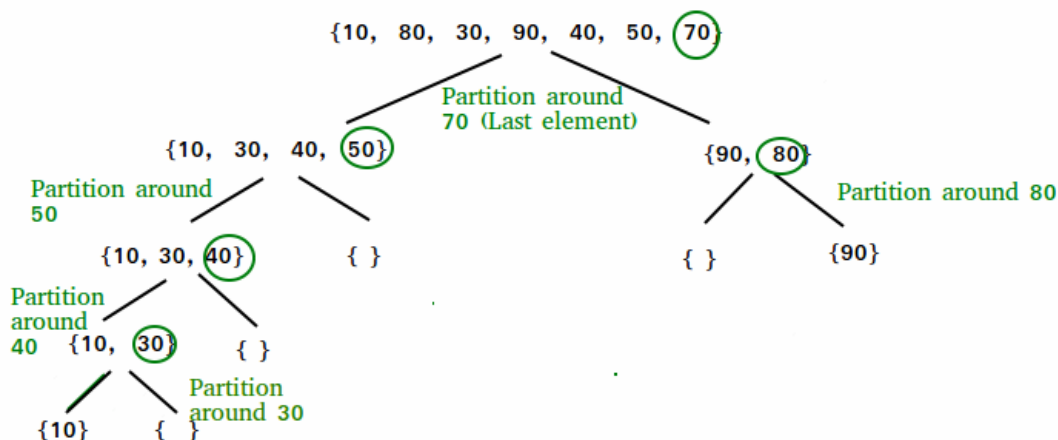
i) **Software:** Turbo C.

4.0) THEORY:

QuickSort is a sorting algorithm based on the [Divide and Conquer algorithm](#) that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.

How does QuickSort work?

The key process in **quickSort** is a **partition()**. The target of partitions is to place the pivot (any element can be chosen to be a pivot) at its correct position in the sorted array and put all smaller elements to the left of the pivot, and all greater elements to the right of the pivot. Partition is done recursively on each side of the pivot after the pivot is placed in its correct position and this finally sorts the array.



How Quicksort works

Choice of Pivot:

There are many different choices for picking pivots.

PREPARED BY:
PROF.A.G.SHARMA

APPROVED BY: (H.O.D.)
DR. A. S. MANEKAR

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : To study and Implement divide and conquer technique			
EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/03		ISSUE NO. : 00	ISSUE DATE : 01.07.2023	
REV. DATE :		REV. NO. :	DEPTT. : INFORMATION TECHNOLOGY	
LABORATORY : Design and Analysis of Algorithm			SEMESTER : VI	PAGE: 2 OF 6

- [Always pick the first element as a pivot.](#)
- Always pick the last element as a pivot (implemented below)
- [Pick a random element as a pivot.](#)
- Pick the middle as the pivot.

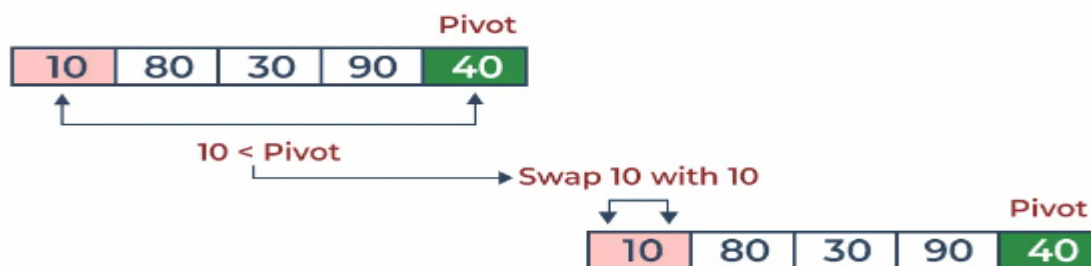
Partition Algorithm:

The logic is simple, we start from the leftmost element and keep track of the index of smaller (or equal) elements as *i*. While traversing, if we find a smaller element, we swap the current element with *arr[i]*. Otherwise, we ignore the current element.

Let us understand the working of partition and the Quick Sort algorithm with the help of the following example:

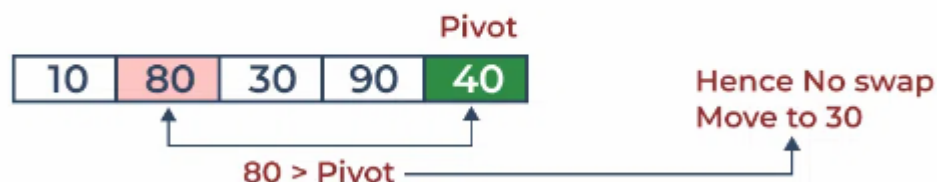
Consider: $arr[] = \{10, 80, 30, 90, 40\}$.

- Compare 10 with the pivot and as it is less than pivot arrange it accordingly.



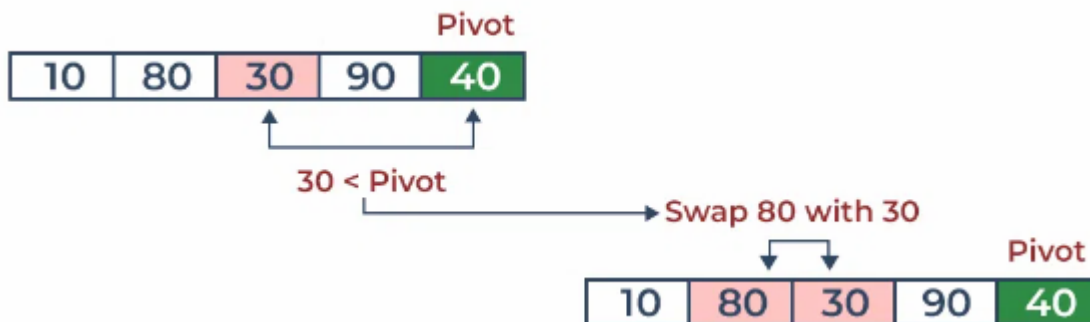
Partition in QuickSort: Compare pivot with 10

- Compare 80 with the pivot. It is greater than pivot.

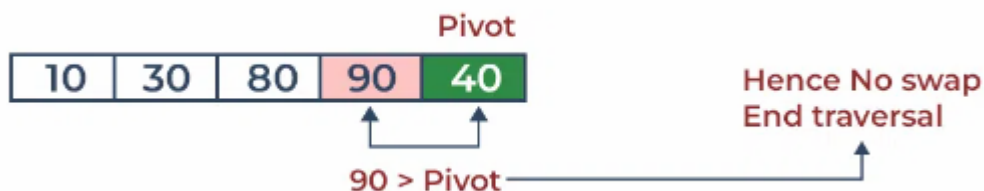


- Compare 30 with pivot. It is less than pivot so arrange it accordingly.

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : To study and Implement divide and conquer technique			
EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/03		ISSUE NO. : 00	ISSUE DATE : 01.07.2023	
REV. DATE :		REV. NO. :	DEPTT. : INFORMATION TECHNOLOGY	
LABORATORY : Design and Analysis of Algorithm			SEMESTER : VI	PAGE: 3 OF 6



- Compare 90 with the pivot. It is greater than the pivot.



- Arrange the pivot in its correct position.

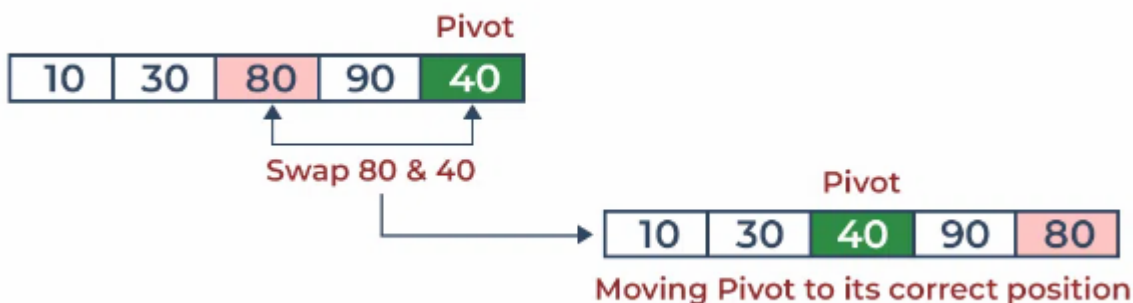
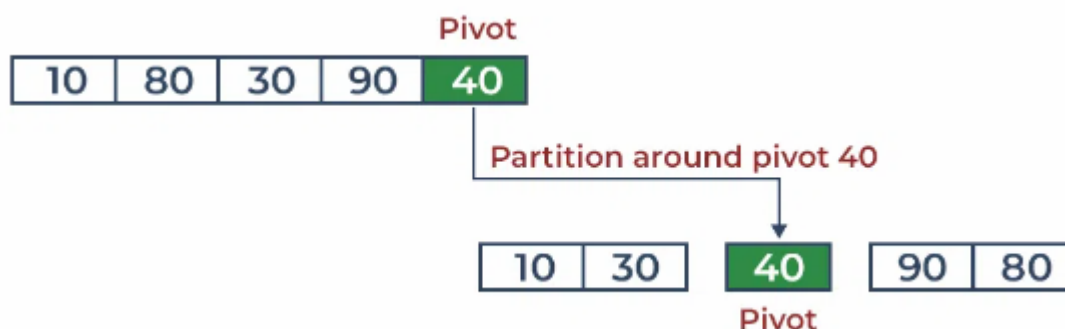


Illustration of Quicksort:

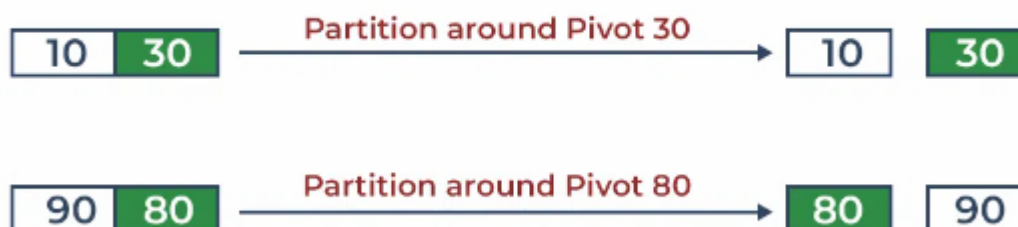
As the partition process is done recursively, it keeps on putting the pivot in its actual position in the sorted array. Repeatedly putting pivots in their actual position makes the array sorted. Follow the below images to understand how the recursive implementation of the partition algorithm helps to sort the array.

- Initial partition on the main array:

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : To study and Implement divide and conquer technique			
EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/03		ISSUE NO. : 00	ISSUE DATE : 01.07.2023	
REV. DATE :		REV. NO. :	DEPTT. : INFORMATION TECHNOLOGY	
LABORATORY : Design and Analysis of Algorithm			SEMESTER : VI	PAGE: 4 OF 6



- Partitioning of the subarrays:



Complexity Analysis of Quick Sort:

Time Complexity:

- Best Case: $\Omega(N \log N)$**
The best-case scenario for quicksort occurs when the pivot chosen at each step divides the array into roughly equal halves. In this case, the algorithm will make balanced partitions, leading to efficient Sorting.
- Average Case: $\theta(N \log N)$**
Quicksort's average-case performance is usually very good in practice, making it one of the fastest sorting Algorithms.
- Worst Case: $O(N^2)$**
The worst-case Scenario for Quicksort occurs when the pivot at each step consistently results in highly unbalanced partitions. When the array is already sorted and the pivot is always chosen as the smallest or largest element. To mitigate the worst-case Scenario, various techniques are used such as choosing a good pivot (e.g., median of three) and using Randomized algorithm (Randomized Quicksort) to shuffle the element before sorting.
- Auxiliary Space: $O(1)$** , if we don't consider the recursive stack space. If we consider the recursive stack space then, in the worst case quicksort could make $O(N)$.

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : To study and Implement divide and conquer technique			
EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/03		ISSUE NO. : 00	ISSUE DATE : 01.07.2023	
REV. DATE :		REV. NO. :	DEPTT. : INFORMATION TECHNOLOGY	
LABORATORY : Design and Analysis of Algorithm			SEMESTER : VI	PAGE: 5 OF 6

Advantages of Quick Sort:

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It has a low overhead, as it only requires a small amount of memory to function.

Disadvantages of Quick Sort:

- It has a worst-case time complexity of $O(N^2)$, which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.
- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions).

Program:

```
#include <stdio.h>
void quicksort (int [], int, int);
int main()
{
    int list[50];
    int size, i;
    printf("Enter the number of elements: ");
    scanf("%d", &size);
    printf("Enter the elements to be sorted:\n");
    for (i = 0; i < size; i++)
    {
        scanf("%d", &list[i]);
    }
    quicksort(list, 0, size - 1);
    printf("After applying quick sort\n");
    for (i = 0; i < size; i++)
    {
        printf("%d ", list[i]);
    }
    printf("\n");

    return 0;
}
void quicksort(int list[], int low, int high)
{
    int pivot, i, j, temp;
```

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : To study and Implement divide and conquer technique			
EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/03		ISSUE NO. : 00	ISSUE DATE : 01.07.2023	
REV. DATE :		REV. NO. :	DEPTT. : INFORMATION TECHNOLOGY	
LABORATORY : Design and Analysis of Algorithm			SEMESTER : VI	PAGE: 6 OF 6

```

if (low < high)
{
    pivot = low;
    i = low;
    j = high;
    while (i < j)
    {
        while (list[i] <= list[pivot] && i <= high)
        {
            i++;
        }
        while (list[j] > list[pivot] && j >= low)
        {
            j--;
        }
        if (i < j)
        {
            temp = list[i];
            list[i] = list[j];
            list[j] = temp;
        }
    }
    temp = list[j];
    list[j] = list[pivot];
    list[pivot] = temp;
    quicksort(list, low, j - 1);
    quicksort(list, j + 1, high);
}
}

```

5.0) Conclusion:

Implemented Quick Sort Algorithm and it's Analysis.